Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Course Title:	Distributed Cloud Computing
Course Number:	COE 892
Semester/Year (e.g.F2016)	W2026

Instructor:	Muhammad Jaseemuddin

Assignment/Lab Number:	1
Assignment/Lab Title:	Concurrency vs Parallelism

Submission Date:	01/02/2026
Due Date:	03/02/2026

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Meng	Alex	501095337	03	A.M

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: http://www.ryerson.ca/senate/current/pol60.pdf

Introduction:

This lab explores Python threading and multiprocessing, using both modules to process the data of several landmine detector robots. The area of land these robots operate in is modeled as a 2D array of zeroes and non-zeros, where non-zeros indicate a mine at that position. The map.txt file holds this map data, along with 2 numbers in the first row, divided by a space, representing the number of rows and columns in the map.

Part 1:

The program consists of 3 functions: start_rovers(), draw_path(), and checkRover().

The program starts by calling start_rovers(), providing an integer as input denoting the number of rovers to run in parallel. Inside this function, a Lock object is defined to control access to the map.txt file, and the rover threads are initialized and started. Each thread runs the draw_path() function.

The draw_path() function reads the map.txt file after obtaining the lock and loads the map data into the relevant variables. The initial position and direction of each rover is (0,0) facing “south” or down. From here, the program enters a loop. First, it calls checkRover() to get a command. checkRover() makes the api call to the server and retrieves a command for the rover. The command can be “R” or “L” to turn right or left, respectively, or “M” to move. If a rover moves while there is a mine beneath it, it sets a flag indicating that a mine was present. The “D” command makes the rover check the map array to see if there is a mine at that location. If there is, it decrements the counter tracking the number of mines remaining and clears the mine from the map. Now, the rover marks its current position in the output array with an asterisk. Finally, the rover checks if the “mine” flag was set. If it is set, it breaks out of the loop and saves the output array with its path drawn out in the relevant text file. If it isn’t set, it checks to see if the number of mines remaining is greater than zero before restarting the loop or continuing on to save the output array. The program exits when all mines on the map have been defused.

The program is designed to run in parallel, but it can be forced to run in sequence, one rover at a time, by simply calling start_rovers with 1 as the input, multiple times, and timing the execution time.

For testing, 10 rovers were run with each execution style, 10 in parallel and 10 in series. With this part, the sequential execution time was slower, while the parallel execution time took less time. These times were somewhat random, since the instruction generation from the servers is randomized. Each run was done on different instances of the same map, containing 2 mines. Provided below is a table of multiple runs and their execution times:

Table 1: Execution times, Parallel vs Sequential

Run 1	**Time**	**Run 2**	**Time**	**Run 3**	**Time**	**Run 4**	**Time**
Parallel	0.79	Parallel	1.09	Parallel	1.08	Parallel	0.69

Series	1.97	Series	1.59	Series	1.24	Series	1.44

Part 2:

This part involves implementing a hashing function as a stand-in for disarming the mines. This function reads the mines.txt file, which is formatted as follows:

1. The first line is a series of space-separated x,y coordinates denoting the positions of mines on the map. The individual x and y values are comma-separated.
2. Each line after that is a serial number corresponding to the mines at the positions listed in the first line of the mines.txt file, in the same order as they appear in the first line.

The digMine() function is called when a “D” command is received. This function takes the current x and y coordinates of the rover as input, and an integer denoting the number of threads to be used to run the disarming function. The digMine() function checks the positions of mines in the mines.txt file against the current position of the rover. If there is a mine at the rover’s current position, the function initializes and starts a number of run_hash() threads corresponding to the number of threads specified as input to the digMine() function. It passes the index of the serial number for this mine, the position, and an event flag to each of these child functions.

Each run_hash() thread does the following:

1. It generates a random pin based on the location of the mine, and a random set of 8 letters and numbers.
2. It creates a temporary key by concatenating the randomly generated pin to the serial number of the mine.
3. This temporary key is sent through the SHA256 hash function.
4. If the resulting hash has 6 or more leading zeroes, the pin is valid, and an event flag is set, causing all other parallel threads to terminate. Immediately after, the mine is cleared from the map, and the position and serial number of the mine are removed from the lists keeping track of them.
5. Otherwise, the loop checks if the event flag is set, and if it isn’t, it restarts.

For testing purposes, the command input was hard-coded to prevent rovers from constantly running across mines without digging them and blowing up. Again, the program was run in both parallel and sequential execution, and execution times were measured. Parallel execution was set to use 10 threads in parallel to handle the hashing, while sequential execution only used one. Both were run on the same map containing 2 mines.

Table 2: Execution times, Parallel vs Sequential

Run 1	**Time (s)**	**Run 2**	**Time (s)**	**Run 3**	**Time (s)**	**Run 4**	**Time (s)**
Parallel	55.47	Parallel	178.3	Parallel	72.16	Parallel	3.36
Series	71.46	Series	256.15	Series	84.03	Series	32.59

The pins are randomly generated, so the run times are quite random, as sometimes, a valid pin is found quickly, while other runs may take a long time. However, the parallel execution is still faster most of the time.

Conclusion

This lab demonstrated the practical use of Python's threading and multiprocessing concepts through a simulated landmine detection and defusal system. In Part 1, multithreading was used to run multiple rover agents concurrently while safely sharing access to a common map using locks. The results showed that parallel execution generally outperformed sequential execution, as multiple rovers were able to explore and defuse mines simultaneously. Although execution times varied due to the randomized nature of server-issued commands, the overall trend confirmed that parallelism reduced total runtime when compared to running rovers sequentially.

Part 2 extended these concepts by introducing a computationally intensive hashing task to simulate mine disarmament. By parallelizing the hashing process across multiple threads, the system was able to significantly reduce the time required to find a valid pin in many cases. While the random pin generation led to large variations in execution time, parallel hashing consistently demonstrated better average performance than a single-threaded approach.

Overall, this lab highlighted both the advantages and challenges of parallel programming. It showed how threading can improve performance for both I/O-bound and CPU-bound tasks, while also emphasizing the importance of synchronization mechanisms and careful design when working with shared resources.